

use in pattern matching with strings, or string matching. The following are some of the notational short-hand symbols used in a regular expression:

- The '.' (dot) is the wild card character that matches any single character.
 - | stands for 'or'. A|b means either A or b.
 - * stands for a range of characters.
 - '?' is used to represent zero or one instance. 'cars?' can represent either car or cars.
 - + represents one or more instances of an entity.
 - * represents zero or more instances of an entity.
 - () is for grouping a sequence of characters.
 - Character classes: [abc] represents a|b|c.
- Given below are the rules for naming an identifier in C::
- It must begin with an underscore or an alphabet.
 - It can be followed by zero or more underscores, a letter of the alphabet or a number.

We can write the regular expression as:

```
[a-zA-Z_](a-zA-Z_0-9)*
```

Intelligent people always find easy ways to get their job done. It may be difficult to repeatedly write 'a-zA-Z' for letters of the alphabet. Hence, the idea emerged to give names to regular expressions that are frequently used and are lengthy.

Regular definitions allow one to give names to regular expressions. These short names help to avoid repeatedly writing lengthy regular expressions.

As an example, l -> a-zA-Z and d -> 0-9. Given this regular definition, a regular expression for identifiers can be re-written as:

```
( l | _ )( l | d | _ )*
```

The structure of a Lex program

To quote from the book of Genesis, where the Biblical version of the creation of the world is described, "And God said, "Let there be light" and there was light." Likewise, the Lex compiler said, "Let there be the file lex.yy.c: and there was lex.yy.c." But wait, before discussing lex.yy.c, we must discuss the structure of a Lex program, which consists of the following sections:

- Definition section
- Rule section
- User defined subroutines

The definition section: The definition section starts with %f and ends with %}. We can write C program segments within this section. Everything contained within this section will be directly copied to the generated C file. Conventions for comments in C can be followed for inserting comments in this section. If we write comments outside %f %}, we must indent them with a single white space for the Lex compiler to understand them as comments.

```
e.g. %{
    /* This is a sample program.
       This will be copied to C file
       This section contains # define, extern, and global
       declarations (if any)

    */
}%
```

This section may also contain regular definitions. But the regular definitions are defined outside %f %}. It is optional to include regular definitions.

The rule section: This section starts and ends with %%. All the rules for translation are defined within this section. Each rule has a pattern and an action part separated by a white space. The patterns are specified using a regular expression. '(a vertical bar)' can be used to separate patterns for which the action is the same.

For example:

```
abc | sdf {printf("strings");}
```

...results in printing 'strings' if the input contains either 'abc' or 'sdf'. The rules are considered in the order in which they are specified. Let me explain this point with the help of an example.

The input contains "int a=10; char name[10]="sharon" " and rules are specified as:

```
%%
[a-zA-Z]+ { printf("strings");}
int[char]break {printf("keywords");}
%%
```

The first rule says that any combination of letters from A to Z or a to z can be identified as a string. So 'a', 'name', and 'sharon' will be identified as strings. Even though 'int' and 'char' are keywords, they too are combinations of letters from a to z, resulting in Lex identifying them as strings. Now let us change the order of the rules:

```
%%
int[char]break {printf("keywords");}
[a-zA-Z]+ { printf("strings");}
%%
```

This set of rules will identify 'int' and 'char' in the input file as keywords and 'a', 'name', and 'sharon' as strings for the same input. Thus the order in which rules are specified does matter in Lex programs.

Another notable feature is that Lex patterns will match a given input character or string only once and execute the action for the longest possible match.

For example, the input contains a string 'intimacy'.